

HORIZON CAMPUS



ITDS11023 - Fundamentals of Programming
year 1, Semester 1,2025

Fuel Cost Tracker

Project Document

U.K JANITHA CHATHURWA-ITBDS-2415-0017
Submission Date - 15.08.2025

FUEL-COST-TRACKER System — Project Report

➤ Introduction

The Fuel Expense Tracker is a console application developed in C. Its main purpose is to help users monitor their fuel expenditure. The user is prompted to enter the fuel purchase date with its corresponding quantity and the price per liter. Its total cost is computed automatically. Users can view all the stored records in a clear table and generate summary reports containing the total and average expenses. Fuel expense records are stored in a text file, which is automatically opened during application startup. As with many such applications, this project illustrates the use of functions, arrays, loops, and conditional statements as well as file handling in C while highlighting modular programming for easier maintenance and improved readability.

➤ Features

1. **Add Fuel Record:** Users can record a fuel purchase by entering the date, the number of liters bought, and the price per liter.
2. **View All Records:** This feature displays all fuel purchase records in a neatly organized table, including the total cost for each entry.
3. **Generate Expense Report:** The system calculates and shows the total and average fuel expenses, providing a quick financial summary.
4. **Persistent Storage:** All records are saved to a file named `fuel_records.txt`, ensuring that data is retained between sessions.
5. **Input Validation:** The system enforces a cap of 100 records, preventing overflow and maintaining data integrity.
6. **User-Friendly CLI:** A clear and interactive menu enhances the usability experience, making navigation simple and intuitive.
7. **Search Records by Date:** Users can quickly retrieve fuel purchase records by entering a specific date.
8. **Monthly Expense Summary:** This feature helps users track their budget by summarizing total spending and calculating the average cost per month.
9. **Fuel Efficiency Tracking (Optional):** Users can log the number of kilometers driven to monitor fuel efficiency in kilometers per liter (km/L).

➤ Source Code

```
#include <stdio.h>

#include <stdlib.h>


#define MAX_RECORDS 100

#define FILENAME "fuel_records.txt"


// Structure to store each fuel purchase record
typedef struct {
    char date[15];
    float liters;
    float pricePerLiter;
    float totalCost;
} Fuel_Records;


// Function prototypes
void add_fuel_Record(Fuel_Records records[], int *count);
void view_fuel_Records(Fuel_Records records[], int count);
void saveToFile(Fuel_Records records[], int count);
void read_fuel_records(Fuel_Records records[], int *count);
float calculate_Total_Expense(Fuel_Records records[], int count);
float calculate_AverageCost(Fuel_Records records[], int count);
void generate_expense_Report(Fuel_Records records[], int count);


int main(void) {
```

```

Fuel_Records records[MAX_RECORDS];

int count = 0;

int choice;


// Load existing records from file
read_fuel_records(records, &count);


// Main menu loop
while (1) {

    printf("\n===== WELCOME TO SRI LANKA FUEL TRACKER
=====\\n");

    printf("1. ?? Add Fuel Record\\n");
    printf("2. ?? View All Records\\n");
    printf("3. ?? Generate Expense Report\\n");
    printf("4. ?? Save to File & Exit\\n");
    printf("Enter your choice (1/2/3/4): ");

    if (scanf("%d", &choice) != 1) {

        printf("Oops! That wasn't a valid choice. Please enter a number from 1 to 4.\\n");
        while (getchar() != '\\n'); // Clear input buffer
        continue;
    }

    switch (choice) {

        case 1:

```

```

        add_fuel_Record(records, &count);

        break;

    case 2:

        view_fuel_Records(records, count);

        break;

    case 3:

        generate_expense_Report(records, count);

        break;

    case 4:

        saveToFile(records, count);

        printf("\n? Data saved successfully. Exiting program...\n");

        exit(0);

    default:

        printf("\nInvalid choice! Please try again.\n");

    }

}

return 0;

}

```

// Adds a new fuel record to the array

```

void add_fuel_Record(Fuel_Records records[], int *count) {

    if (*count >= MAX_RECORDS) {

        printf("\nStorage full! Cannot add more records.\n");

        return;
    }
}

```

```
}
```

```
printf("\nEnter date for fuel addition (DD/MM/YYYY): ");
```

```
scanf("%14s", records[*count].date);
```

```
printf("Enter liters purchased: ");
```

```
scanf("%f", &records[*count].liters);
```

```
printf("Enter price per liter: ");
```

```
scanf("%f", &records[*count].pricePerLiter);
```

```
// Calculate total cost
```

```
records[*count].totalCost = records[*count].liters * records[*count].pricePerLiter;
```

```
printf("Total cost: %.2f\n", records[*count].totalCost);
```

```
(*count)++;
```

```
printf("? Record added successfully!\n");
```

```
}
```

```
// Displays all fuel records in a formatted table
```

```
void view_fuel_Records(Fuel_Records records[], int count) {
```

```
    int i;
```

```
    if (count == 0) {
```

```
        printf("\nNo records to display.\n");
```

```
        return;
```

```
    }
```

```
    printf("\n%-5s %-15s %-10s %-15s %-10s\n", "No.", "Date", "Liters", "Price/Liter",  
    "Total Cost");
```

```
    printf("-----\n");
```

```
    for (i = 0; i < count; i++) {
```

```
        printf("%-5d %-15s %-10.2f %-15.2f %-10.2f\n",
```

```
            i + 1, records[i].date, records[i].liters, records[i].pricePerLiter,  
            records[i].totalCost);
```

```
    }
```

```
}
```

```
// Saves all records to a text file
```

```
void saveToFile(Fuel_Records records[], int count) {
```

```
    FILE *fp = fopen(FILENAME, "w");
```

```
    int i;
```

```
    if (!fp) {
```

```
        printf("\n? Error saving data file!\n");
```

```
        return;
```

```
    }
```

```
    for (i = 0; i < count; i++) {
```

```
        fprintf(fp, "%s %.2f %.2f %.2f\n",
```

```
            records[i].date,
```

```
            records[i].liters,
```

```
            records[i].pricePerLiter,
```

```
            records[i].totalCost);
```

```
    }
```



```

    fclose(fp);
}

// Loads existing records from a text file
void read_fuel_records(Fuel_Records records[], int *count) {
    FILE *fp = fopen(FILENAME, "r");
    if (!fp) {
        printf("No existing data file found. Starting fresh.\n");
        return;
    }

    while (fscanf(fp, "%14s %f %f %f",
        records[*count].date,
        &records[*count].liters,
        &records[*count].pricePerLiter,
        &records[*count].totalCost) == 4) {
        (*count)++;
        if (*count >= MAX_RECORDS) break;
    }
    fclose(fp);
}

// Calculates the total fuel expense
float calculate_Total_Expense(Fuel_Records records[], int count) {
    float total = 0;

```

```

    int i;

    for (i = 0; i < count; i++) {
        total += records[i].totalCost;
    }

    return total;
}

// Calculates the average cost per fuel purchase
float calculate_AverageCost(Fuel_Records records[], int count) {
    if (count == 0) return 0;

    return calculate_Total_Expense(records, count) / count;
}

// Generates a summary report of fuel expenses
void generate_expense_Report(Fuel_Records records[], int count) {
    if (count == 0) {
        printf("\nNo data to generate report.\n");
        return;
    }

    printf("\n===== Fuel Expense Report =====\n");
    printf("Total number of purchases: %d\n", count);
    printf("Total expense: %.2f\n", calculate_Total_Expense(records, count));
    printf("Average cost per purchase: %.2f\n", calculate_AverageCost(records, count));
}

```